

Lecture-8

IF() Function

- The `IF()` function is a simple tool used for validating a condition in MySQL.

How it Works (The Logic):

- **Syntax:** `IF(condition, value_if_true, value_if_false)` .
- **Purpose:** It evaluates the `condition` (an expression).
- If the condition is **TRUE**, it returns `value_if_true` .
- If the condition is **FALSE**, it returns `value_if_false` .
- **Output:** The values returned (`value_if_true` or `value_if_false`) can be either **numeric or strings**.
- **Real-World Example:** If a student's `exam_score` is 50 or above (the condition), the function returns "Pass"; otherwise, it returns "Fail".

ISQL Triggers

- A trigger is an automated set of instructions that runs in response to specific events happening on a database table or view.

Purpose and Activation

1, **Core Function:** Triggers help maintain **** data consistency**** and **enforce business rules** by executing necessary logic whenever data changes occur.

2. **Activation Events:** Triggers are activated by **Data Manipulation Language (DML)** operations:

- `INSERT` (adding new data).
- `UPDATE` (modifying existing data).
- `DELETE` (removing data).

3. **Activation Time:** Triggers activate either **BEFORE** or **AFTER** the associated database event takes place.

- *Example:* You can define a trigger to run automatically *before* a new row is inserted into a table.

- *Business Rule Example:* Charging a £10 overdraft fee if an account balance is less than £500 after a withdrawal transaction.

Types of MySQL Triggers

For every table, you can define six types of triggers:

Time of Action	Trigger Event	Description
BEFORE	INSERT	Activates right before a new row is added.
BEFORE	UPDATE	Activates right before an existing row is modified.
BEFORE	DELETE	Activates right before a row is removed.
AFTER	INSERT	Activates immediately after a new row is successfully added.
AFTER	UPDATE	Activates immediately after a row is successfully modified.
AFTER	DELETE	Activates immediately after a row is successfully removed.

Trigger Actions

- **(Cross-Table Action):** If a university adds a new student (INSERT event), an **AFTER INSERT** trigger can automatically insert a welcome message into a separate **Notifications** table.
- **AFTER DELETE Example (Data Consistency):** If a student's record is deleted from the **Students** table, an **AFTER DELETE** trigger can automatically remove their corresponding record from the **Library_Membership** table.
- **BEFORE INSERT Example (Data Scrubbing/Modification):** A **BEFORE INSERT** trigger can check the value being inserted and modify it if needed, such as converting any average marks less than 0 to 0 before the data is saved.
- **BEFORE UPDATE Example (Enforcing Limits):** A **BEFORE UPDATE** trigger can enforce a rule that no employee can receive a raise greater than 10%. If a user tries to update a salary by 20%, the trigger intercepts the change and automatically caps the new salary value at a 10% increase.

Trigger (OLD vs. NEW)

- When writing the code inside a trigger, you have special access to the data that is being changed.

- **old.<column name>**: Provides the value of a column *before* the DML statement (INSERT, UPDATE, or DELETE) is executed.
- **new.<column name>**: Provides the value of that column *after* the DML statement is executed (or the value being proposed for insertion/update).

Availability of OLD and NEW Values:

Trigger Type	OLD Values Available?	NEW Values Available?
UPDATE	Yes (The original value)	Yes (The proposed new value)
INSERT	No	Yes (The value being inserted)
DELETE	Yes (The value being deleted)	No

Trigger Syntax

- The statement used to create a trigger requires you to
 - specify the name
 - time (BEFORE or AFTER)
 - operation (INSERT, UPDATE, or DELETE)
 - table it belongs to

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name FOR EACH ROW
trigger_body;
```

- For multiple triggers:

```
CREATE TRIGGER trigger_name trigger_time trigger_event
ON table_name FOR EACH ROW
BEGIN
    --variable declarations
    --trigger code
END;
```